

Task1 : Planning

Recorded audio/video streaming (Secure Adaptive Streaming)

End-to-End Architecture & Solution Approach

Prepared By

Md. Masud Rana

Mobile: 01575634380

Email: masudrana.mwp@gmail.com

Portfolio: masud-rana-pro.github.io/portfolio

GitHub: github.com/masud-rana-pro

LinkedIn: linkedin.com/in/masudmwp

Submitted to

HR Department, Rushda Soft

Table of Contents

Executive Summary.....	3
Objective.....	3
End-to-End Pipeline	4
Full Architecture Diagram	5
Requirement-by-Requirement Solution	6,7
Technology Stack Summary	7
Known Limitations / Trade-offs.....	8
Conclusion.....	8

Executive Summary

The Streaming feature has five requirements:

- I. Smooth playback
- II. Near-instant load
- III. Protection against capture/download
- IV. Traceability after a leak
- V. Quality that adapts to network and device

This proposal covers the full lifecycle of a video/audio streaming. The five techniques work together those are:

- A. Direct-to-storage upload + async encoding pipeline:-- keeps ingest fast and the app server stateless
- B. Adaptive Bitrate Streaming (HLS/DASH) :--removes buffering and matches quality to network/screen
- C. CDN edge caching :-- makes start-up near-instant and absorbs the bulk of delivery traffic
- D. Multi-DRM encryption (Widevine + FairPlay + PlayReady) + output protection (HDCP, secure audio path) :-- the preventive controls against capture, recording (video and audio) and download
- E. Server-side A/B forensic watermarking :-- the detective control that traces a leak back to its source session

Objective

This document explains how a recorded audio/video streaming feature can be designed end-to-end from upload through encoding, protection, delivery, and playback so that content plays without buffering, starts almost instantly, is protected from screenshotting/recording/downloading, remains traceable if it leaks, and adapts to the viewer's network speed and screen size.

End-to-End Pipeline

The table below shows end-to-end pipeline that follows large-scale platforms like YouTube, Netflix.

Stage	What happens
1. Upload	The client uploads large video files directly to object storage using a temporary pre-signed URL. The backend only manages metadata and triggers the encoding process.
2. Encoding / Transcoding	Uploaded videos are processed by encoder(Kafka/SQS) workers in the background. They create multiple qualities (e.g. 240p, 480p, 720p, 1080p), thumbnails, subtitles, and HLS/DASH segments.
3. Storage	Raw and processed videos are stored in scalable object storage(AWS S3/GCS/Azure Blob Storage), while the database keeps only metadata and file references.
4. Protection (DRM + Watermark)	Encoded segments are encrypted (CENC for Widevine/PlayReady, cbcs for FairPlay) and, for premium content, passed through the watermark pre-processor to generate A/B variants before being pushed to the CDN.
5. CDN Delivery	The CDN serves video segments from locations close to viewers, reducing loading time and server load.
6. Playback	The player downloads the manifest (.m3u8/.mpd), starts on a low-bitrate segment for a fast start, then continuously adapts quality to bandwidth and screen size all over ordinary HTTP, so no specialized streaming server process is needed.
7. Scale	Microservices (upload / encoding / metadata / streaming API) scale independently behind load balancers; services stay stateless; databases are sharded/NoSQL for write scale; Redis/Memcached caches hot metadata; multi-region deployment plus DNS-based routing sends each viewer to the nearest region.

Full Architecture Diagram

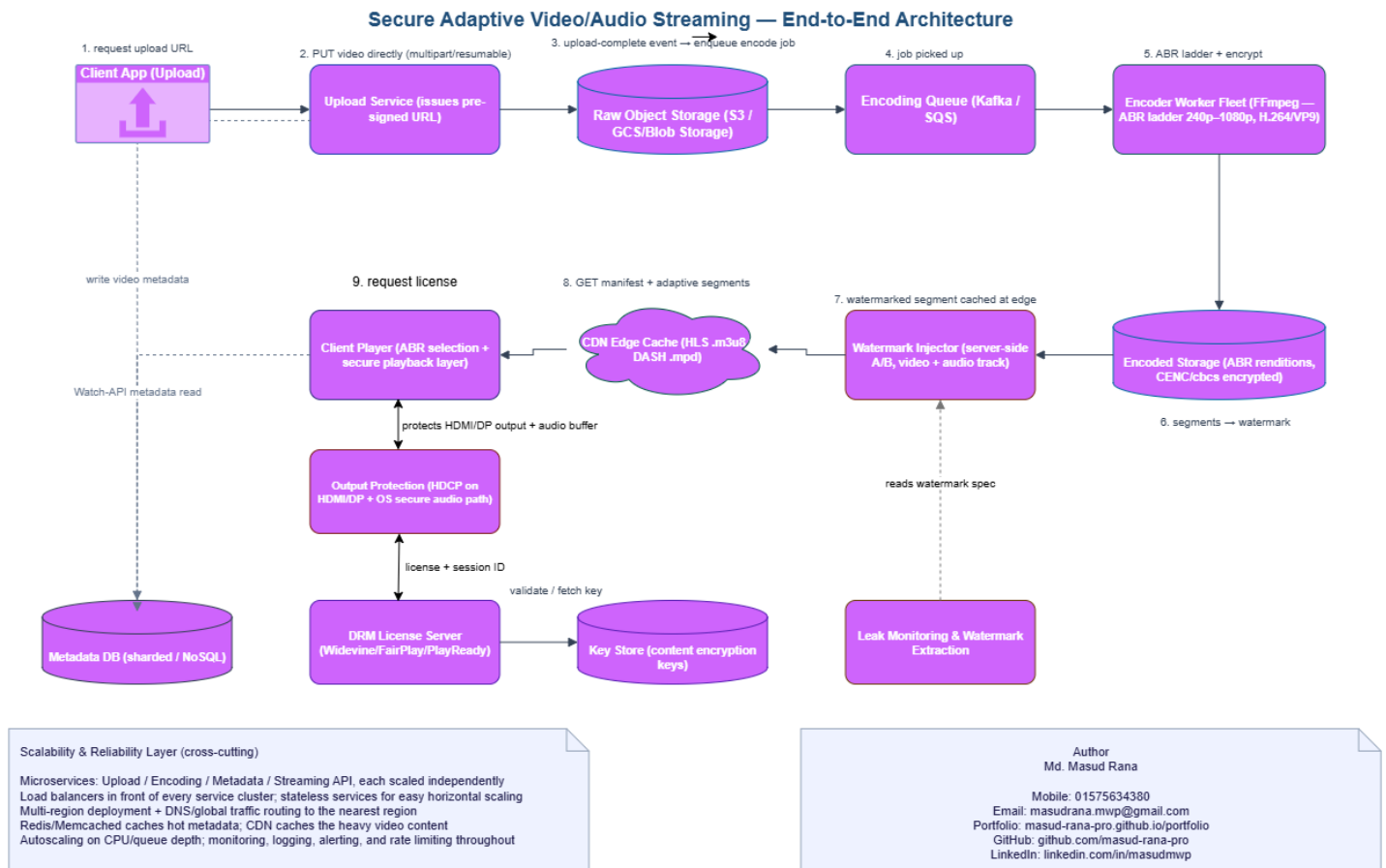


Figure 1: End-to-end flow upload, encode, protect (DRM + watermark), deliver (CDN), and play (ABR)

Flow summary: The client uploads directly to object storage via a pre-signed URL (bypassing the app server). An encoding job is queued and processed by worker nodes into an ABR ladder. Encoded segments are encrypted and, for premium content, watermarked with a unique per-session A/B pattern before being cached at the CDN edge. The player requests a DRM license, then pulls the manifest and adaptive segments from the edge. If a copy ever leaks, watermark extraction traces it back to the session that requested the license.

Requirement-by-Requirement Solution

Requirement	How it is achieved
No buffering during playback	Adaptive Bitrate Streaming (HLS/DASH): During encoding, an encoder worker fleet produces an ABR ladder (240p-1080p) using H.264/VP9, split into a few seconds per segment. The player continuously measures throughput and switches renditions between segments, so playback never stalls waiting for a single fixed-bitrate stream.
Content loads almost instantly	Direct-to-storage upload (pre-signed URLs) keeps the ingest path light, and CDN edge caching keeps the delivery path fast: Manifests and opening segments are cached close to the viewer, and the player prefetches the default rendition's first segments before play is pressed.
Protected from screenshot / video recording / download	Multi-DRM (Widevine + FairPlay + PlayReady): Every segment is packaged in the encryption mode each system needs CENC (AES-CTR) for Widevine/PlayReady, cbcs (AES-CBC) for FairPlay and the content key is only released, inside an isolated Content Decryption Module, after the license server verifies the session. The key never touches JavaScript or normal device memory. On native apps this is reinforced by the OS secure-video path (FLAG_SECURE / secure texture), which blocks screenshot and screen-record APIs. <i>Browser playback cannot be made 100% screenshot-proof against out-of-browser capture tools stated honestly as a limitation, compensated by watermarking.</i>
Protected from audio recording	Audio is a separate attack surface from screen recording and needs its own controls: (a) HDCP (High-bandwidth Digital Content Protection) on HDMI/DisplayPort output blocks capture of the audio-video signal leaving the device through an external port. (b) The OS secure audio path marks the decoded audio buffer as protected, which blocks system-level loopback recording and virtual-audio-cable capture on platforms that support it (notably Windows and Android). (c) Tudio can carry its own inaudible forensic watermark alongside the video watermark, using the same session-identifier logic, so an audio-only leak (e.g. a phone recording sound near a speaker) is still traceable. As with screenshots, no software control stops a physical microphone recording open air sound that residual risk is covered by watermarking, not prevention.
Traceable if content leaks	Server-side A/B forensic watermarking (the ETSI TS 104 002 pattern): The watermark pre-processor produces two near-identical, invisibly-different variants (A/B) of every segment applied to both

	the video and the audio track. The CDN edge serves each viewer a unique A/B sequence chosen from their session token that sequence is the viewer's traceable identifier. The mark rides through re-encoding, cropping and even screen/audio recording, so if a leaked copy is found, extraction recovers the identifier and maps it back to the exact session. This is a detective control, not a preventive one it changes a leak's consequence from anonymous to attributable, and it is run alongside DRM, never instead of it.
Loads according to internet speed and screen size	The ABR ladder covers multiple resolutions: The player selects a rendition based on measured bandwidth and the device's screen resolution, so a phone on 3G never pulls a 1080p segment it cannot display, while a laptop on Wi-Fi is not capped at low quality.

Technology Stack Summary

Layer	Technology
Upload	Pre-signed URLs to object storage; resumable multipart upload
Encoding / ABR packaging	FFmpeg -> HLS (.m3u8) and/or MPEG-DASH (.mpd) ladder, queued via Kafka/SQS
Storage	Object storage (AWS S3/GCS/Azure Blob Storage), for media; sharded/NoSQL DB for metadata only
Delivery	CDN with edge caching (CloudFront / Cloudflare / Akamai) or a proprietary CDN at scale
Content protection	Multi-DRM: Widevine + FairPlay + PlayReady, via a DRM/license service
Leak traceability	Server-side A/B forensic watermarking (ETSI TS 104 002) + monitoring service
Client playback	HLS.js / Shaka Player / native ExoPlayer (Android) / AVPlayer (iOS)
Scalability	Stateless microservices, load balancers, autoscaling, Redis/Memcached, multi-region routing

Known Limitations / Trade-offs

- I. Browser-based playback cannot fully block screen recording (capture software works outside the browser's control) and DRM raises the effort required, and watermarking provides the fallback: attribution after the fact.
- II. Multi-DRM licensing adds infrastructure cost and three separate license-server integrations; a smaller rollout could start with Widevine only (largest device share) and add FairPlay/PlayReady later.
- III. A/B watermarking roughly doubles CDN cache footprint on marked content and lowers hit ratio to mark only contractually-required content, not the full catalogue.
- IV. Very short ABR segments (2-4s) adapt faster to network changes but increase request overhead; 6-10s segments are more CDN-cache-friendly that a tunable choice, not fixed.

Conclusion

Together, direct upload + async encoding + CDN caching solve the performance requirements (no buffering, near-instant start, network/screen-aware quality). Multi-DRM and server-side forensic watermarking form two complementary layers of content protection one preventive, one detective addressing the security requirements, while a stateless, load-balanced, multi-region service layer keeps the whole system reliable at scale.